Fundamentals of Programming I

Chapter 2 - Basic Programming Constructors Coding Exercises - Solutions

Try each exercise first. Many correct programs or solutions are exist, and yours may differ and still be right.

Easy level

Exercise 1: Escape sequences

```
#include <iostream>
using namespace std;

int main() {
    cout << "Name:\tAbebe" << endl;
    cout << "She said \"Hello\"" << endl;
    return 0;
}
```

- `\t` prints a tab.
- `\"` prints a double quote inside a string.

Sample run:

```
Name:    Abebe
She said "Hello"
```

Exercise 2: Sum, difference and product

```
#include <iostream>
using namespace std;

int main() {
```

```
    int a, b;
    cout << "Enter two integers: ";
    cin >> a >> b;
    cout << "Sum: " << a + b << endl;
    cout << "Difference: " << a - b << endl;
    cout << "Product: " << a * b << endl;
    return 0;
}
```

- `cin >> a >> b;` reads both numbers.
- Arithmetic can be done directly inside the `cout` chain.

Sample run:

```
Enter two integers: 7 3
Sum: 10
Difference: 4
Product: 21
```

Exercise 3: Rectangle area and perimeter

```
#include <iostream>
using namespace std;

int main() {
    double length, width;
    cout << "Length and width: ";
    cin >> length >> width;
    cout << "Area: " << length * width << endl;
    cout << "Perimeter: " << 2 * (length + width) << endl;
    return 0;
}
```

- Use `double`, not `int`, so decimals are not cut off.

Sample run:

```
Length and width: 5.5 2
Area: 11
Perimeter: 15
```

Exercise 4: Simple interest

```
#include <iostream>
using namespace std;

int main() {
    double p, r, t;
    cout << "Principal, rate, time: ";
    cin >> p >> r >> t;
    double interest = p * r * t / 100;
    cout << "Simple interest: " << interest << endl;
    return 0;
}
```

- Declare a variable for the result and give it a clear name.
- Multiplying first and dividing last keeps the formula simple.

Sample run:

```
Principal, rate, time: 2000 5 3
Simple interest: 300
```

Exercise 5: Average of three numbers

```
#include <iostream>
using namespace std;

int main() {
    int a, b, c;
    cout << "Enter three integers: ";
    cin >> a >> b >> c;
    double average = (a + b + c) / 3.0;
    cout << "Average: " << average << endl;
    return 0;
}
```

- Dividing by `3.0` (a real number) forces real division.
- With `/ 3` the division would be an integer division and give 6.

Sample run:

```
Enter three integers: 5 6 8
Average: 6.33333
```

Medium level

Exercise 6: Seconds to hours, minutes, seconds

```
#include <iostream>
using namespace std;

int main() {
    int total;
    cout << "Seconds: ";
    cin >> total;
    int hours = total / 3600;
    int minutes = (total % 3600) / 60;
    int seconds = total % 60;
    cout << hours << " h " << minutes << " m " << seconds << " s" << endl;
    return 0;
}
```

- `/` gives whole hours; `%` gives what is left over.
- Remaining seconds after the hours: `total % 3600`. Divide that by 60 for minutes.

Sample run:

```
Seconds: 3725
1 h 2 m 5 s
```

Exercise 7: Largest of three (conditional operator)

```
#include <iostream>
using namespace std;

int main() {
    int a, b, c;
    cin >> a >> b >> c;
    int big = (a > b) ? ((a > c) ? a : c)
```

```
                      : ((b > c) ? b : c);
    cout << "Largest: " << big << endl;
    return 0;
}
```

- First decide the winner of a and b, then compare that winner with c.
- Conditional operators can be nested inside each other.

Sample run:

```
4 9 6
Largest: 9
```

Exercise 8: Swap two numbers

```
#include <iostream>
using namespace std;

int main() {
    int x, y, temp;
    cin >> x >> y;
    cout << "Before: " << x << " " << y << endl;
    temp = x;
    x = y;
    y = temp;
    cout << "After:  " << x << " " << y << endl;
    return 0;
}
```

- Like swapping the contents of two cups: you need a third, empty cup (`temp`).
- The order matters: save x in temp first, otherwise it is overwritten.

Sample run:

```
10 25
Before: 10 25
After:  25 10
```

Exercise 9: BMI with formatted output

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double weight, height;
    cin >> weight >> height;
    double bmi = weight / (height * height);
    cout << fixed << setprecision(2);
    cout << setw(12) << "Weight" << setw(12) << "Height" << setw(12) << "BMI" << endl;
    cout << setw(12) << weight << setw(12) << height << setw(12) << bmi << endl;
    return 0;
}
```

- Include `<iomanip>` for `setw` and `setprecision`.
- `setw` affects only the next value, so repeat it for each column.
- `fixed` with `setprecision(2)` prints exactly two decimals.

Sample run:

```
70 1.75
      Weight      Height         BMI
       70.00        1.75       22.86
```

Exercise 10: Sum of the digits of a 3-digit number

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    int hundreds = n / 100;
    int tens = (n / 10) % 10;
    int ones = n % 10;
    cout << "Digit sum: " << hundreds + tens + ones << endl;
    return 0;
}
```

- `n % 10` gives the last digit.
- `n / 100` gives the first digit (integer division drops the rest).

- `(n / 10) % 10` gives the middle digit.

Sample run:

```
482
Digit sum: 14
```

Hard level

Exercise 11: Leap year

```
#include <iostream>
using namespace std;

int main() {
    int year;
    cin >> year;
    bool leap = ((year % 4 == 0) && (year % 100 != 0)) || (year % 400 == 0);
    cout << (leap ? "Leap year" : "Not leap year") << endl;
    return 0;
}
```

- The rule is written directly as a logical expression with `&&` and `||`.
- Test the special cases: 1900 is divisible by 100 but not 400, so it is not a leap year.

Sample run:

```
1900
Not leap year
```

Exercise 12: Reverse a 3-digit number

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
```

```
    int reversed = (n % 10) * 100 + ((n / 10) % 10) * 10 + n / 100;
    cout << "Reversed: " << reversed << endl;
    return 0;
}
```

- Take the last digit and give it the hundreds place, keep the middle digit in the tens place, and move the first digit to the ones place.
- For 120 the result is 021, but an integer does not store a leading zero, so it prints 21.

Sample run:

```
123
Reversed: 321
```

Exercise 13: Distance between two points

```
#include <iostream>
#include <cmath>
#include <iomanip>
using namespace std;

int main() {
    double x1, y1, x2, y2;
    cin >> x1 >> y1 >> x2 >> y2;
    double dx = x2 - x1, dy = y2 - y1;
    double distance = sqrt(pow(dx, 2) + pow(dy, 2));
    cout << fixed << setprecision(2) << "Distance: " << distance << endl;
    return 0;
}
```

- `sqrt()` and `pow()` come from `<cmath>`.
- Computing `dx` and `dy` first keeps the formula readable.

Sample run:

```
1 2 4 6
Distance: 5.00
```

Exercise 14: Wrapping around

```cpp
#include <iostream>
using namespace std;

int main() {
    unsigned short n = 65535;
    cout << "Start:   " << n << endl;
    n = n + 1;
    cout << "After +1: " << n << endl;
    n = n - 1;
    cout << "After -1: " << n << endl;
    return 0;
}
```

- An unsigned short holds 0 to 65535. Going past the maximum wraps to 0, and going below 0 wraps to 65535.
- C++ gives no warning. This is the overflow idea from the notes.

Sample run:

```
Start:   65535
After +1: 0
After -1: 65535
```

Exercise 15: Bitwise operators

```cpp
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cin >> a >> b;
    cout << "a & b  = " << (a & b) << endl;
    cout << "a | b  = " << (a | b) << endl;
    cout << "a ^ b  = " << (a ^ b) << endl;
    cout << "~a     = " << (~a) << endl;
    cout << "a << 2 = " << (a << 2) << endl;
    cout << "a >> 1 = " << (a >> 1) << endl;
    cout << (((a & 1) ? "odd" : "even")) << endl;
    return 0;
}
```

- Put brackets around bitwise expressions inside `cout`, because `<<` also means output.

- Shifting left by 2 multiplies by 4; shifting right by 1 divides by 2.
- The last bit of an even number is 0, so `a & 1` is 0 (false) for even numbers.

Sample run:

```
8 3
a & b  = 0
a | b  = 11
a ^ b  = 11
~a     = -9
a << 2 = 32
a >> 1 = 4
even
```

Part 2: Operator precedence, solutions

Compare your steps with these. The method matters as much as the final answer.

A. Add brackets (no calculation)

Question 1 Easy

```
n <= p && n >= p || n == 0
```

Fully bracket the expression to show the order of evaluation.

Steps: Relational operators (<= and>=) first; then && before ||.

Answer: `(((n <= p) && (n >= p)) || (n == 0))`

Question 2 Easy

```
a + b * c - d / e % f
```

Fully bracket the expression.

Steps: *, / and % come before + and -. / and % are on the same level, so left to right: (d / e) % f. Then + and - left to right.

Answer: `((a + (b * c)) - ((d / e) % f))`

Question 3 Medium

```
x += y = z * 2
```

Fully bracket the expression.

Steps: * first. Assignment operators are right to left, so y = (z * 2) happens before x += ...

Answer: `x += (y = (z * 2))`

Question 4 Medium

```
!a == b && c
```

Fully bracket the expression.

Steps: ! (unary) is higher than ==, and == is higher than &&.

Answer: `(((!a) == b) && c)`

Question 5 Hard

```
a = b > c ? d : e + 1
```

Fully bracket the expression.

Steps: Relational > and + first. ?: is lower than those, and = is lowest, so it takes the whole conditional as its value.

Answer: `a = ((b > c) ? d : (e + 1))`

B. Work out the result by hand

Question 6 Easy

```
int r = 2 + 3 * 4;
```

Find r.

Steps: 3 * 4 = 12, then 2 + 12 = 14.

Answer: `r = 14`

Question 7 Easy

```
int r = 20 - 6 / 2 + 1;
```

Find r.

Steps: 6 / 2 = 3. Then left to right: 20 - 3 = 17, 17 + 1 = 18.

Answer: `r = 18`

Question 8 Easy

```
int r = 17 % 5 * 2;
```

Find r.

Steps: % and * are on the same level, so left to right: 17 % 5 = 2, then 2 * 2 = 4.

Answer: `r = 4`

Question 9 Easy

```
int r = 10 / 4 * 2;
```

Find r.

Steps: Same level, left to right. 10 / 4 is integer division = 2 (not 2.5). Then 2 * 2 = 4.

Answer: `r = 4`

Question 10 Easy

```
int r = 3 + 4 > 6;
```

Find r.

Steps: + is higher than >. 3 + 4 = 7, then 7 > 6 is true, which is 1.

Answer: `r = 1`

Question 11 Medium

```
int a = 5, b = 3;
a += b * 2;
```

Find a and b.

Steps: Assignment is lowest, so the right side is calculated first: b * 2 = 6. Then a = 5 + 6 = 11. b is unchanged.

Answer: `a = 11, b = 3`

Question 12 Medium

```
int x = 10, y = 4;
x -= y + 2;
```

Find x and y.

Steps: y + 2 = 6 first. Then x = 10 - 6 = 4. (It is not (x -= y) + 2.)

Answer: `x = 4, y = 4`

Question 13 Medium

```
int a = 12, b = 5;
a %= b + 1;
```

Find a.

Steps: b + 1 = 6 first. Then a = 12 % 6 = 0.

Answer: `a = 0`

Question 14 Medium

```
int a = 3, b = 7, c = 5;
int r = a > b || c == 5 && b > a;
```

Find r.

Steps: && is higher than ||, so it is a > b || (c == 5 && b > a). a > b is 0. c == 5 is 1 and b > a is 1, so 1 && 1 = 1. Then 0 || 1 = 1.

Answer: `r = 1`

Question 15 Medium

```
int x = 5;
int r = !x > 3;
```

Find r.

Steps: ! is higher than >. !x: x is non-zero (true), so !x = 0. Then 0 > 3 is false = 0. (It is not !(x > 3).)

Answer: `r = 0`

Question 16 Medium

```
int a = 5;
a += a > 3;
```

Find a.

Steps: a > 3 is true, which is 1. Then a = 5 + 1 = 6.

Answer: `a = 6`

Question 17 Hard

```
int x = 4, y = 9;
x += y > 5 && x < 10;
```

Find x.

Steps: Right side first: y > 5 is 1; x < 10 is 1; 1 && 1 = 1. Then x = 4 + 1 = 5.

Answer: `x = 5`

Question 18 Hard

```
int a = 6, b = 4;
a -= b *= 2;
```

Find a and b.

Steps: Assignments go right to left. b *= 2 makes b = 8 (value 8). Then a = 6 - 8 = -2.

Answer: `a = -2, b = 8`

Question 19 Hard

```
int a = 5, b = 2;
a *= b += 3;
```

Find a and b.

Steps: Right to left. b += 3 makes b = 5. Then a = 5 * 5 = 25.

Answer: `a = 25, b = 5`

Question 20 Hard

```
int a = 5, b = 10, c = 15;
int r = a + b > c || !(c - b == a);
```

Find r.

Steps: a + b = 15, and 15 > 15 is 0. In the brackets, c - b = 5 and 5 == 5 is 1, so !1 = 0. Then 0 || 0 = 0.

Answer: `r = 0`

Question 21 Hard

```
int x = 7;
x %= 4 + 1;
x *= x > 1;
```

Find the final x.

Steps: Line 2: 4 + 1 = 5, so x = 7 % 5 = 2. Line 3: x > 1 is 2 > 1 = 1, so x = 2 * 1 = 2.

Answer: `x = 2`

Question 22 Hard

```
int a = 1, b = 2, c = 3;
int r = a == 1 || b++ > 5 && c--;
```

Find r, b and c.

Steps: && is higher than ||, so it is (a == 1) || ((b++ > 5) && c--). The left side a == 1 is true, so || does not evaluate the right side at all (short-circuit). b++ and c-- never run.

Answer: `r = 1, b = 2, c = 3`

Question 23 Hard

```
int x = 5, y = 0;
int r = x > 3 && y++ < 1;
```

Find r and y.

Steps: x > 3 is true, so the right side is evaluated. y++ gives the old value 0 (and then y becomes 1). 0 < 1 is true. So 1 && 1 = 1.

Answer: `r = 1, y = 1`

Question 24 Hard

```
int x = 3, y = 5;
x += y;
y -= x;
x *= y;
```

Find the final x and y.

Steps: Line 1: x = 3 + 5 = 8. Line 2: y = 5 - 8 = -3. Line 3: x = 8 * (-3) = -24.

Answer: `x = -24, y = -3`

Question 25 Hard

```
int a = 7;
int r = a > 5 ? a *= 2 : a /= 2;
```

Find r and a.

Steps: The condition a > 5 is true, so only the first branch runs: a *= 2 makes a = 14, and the value 14 goes to r. The branch a /= 2 is never evaluated.

Answer: `r = 14, a = 14`